PROYECTO ACADEMICO — INGENIERIA ELECTRICA — CUC

Tutores On-Line

Plataforma web para conectar estudiantes con tutores calificados

Entrega — Corte academico

Fecha de entrega: 13 de octubre de 2026

Repositorio: github.com/jgamerot07-prog/TUTORES_ONLINE_FINAL

1. Integrantes del equipo

Pendiente de confirmar: faltan correo y rol de Juan Sampayo, y posibles integrantes adicionales. Completa la tabla si hace falta antes de entregar el documento.

#	Nombre completo	Correo institucional	Rol en el proyecto
1	Juan Illidge	jillidge37@gmail.com	(completar)
2	Juan Sampayo	_____	(completar)
3	_____	_____	(completar)

2. URL del repositorio

Repositorio GitHub: https://github.com/jgamerot07-prog/TUTORES_ONLINE_FINAL

Rama de trabajo: claude/adoring-darwin-3xn0un

El repositorio contiene codigo frontend (HTML/CSS/JS), backend (API REST en Node.js/Express), scripts SQL de Oracle, diagramas, documentacion e instrucciones de ejecucion. No contiene contraseñas, tokens ni archivos `.env` con credenciales reales (estan excluidos via `.gitignore`, y se provee `.env.example` como plantilla).

3. URL del hosting

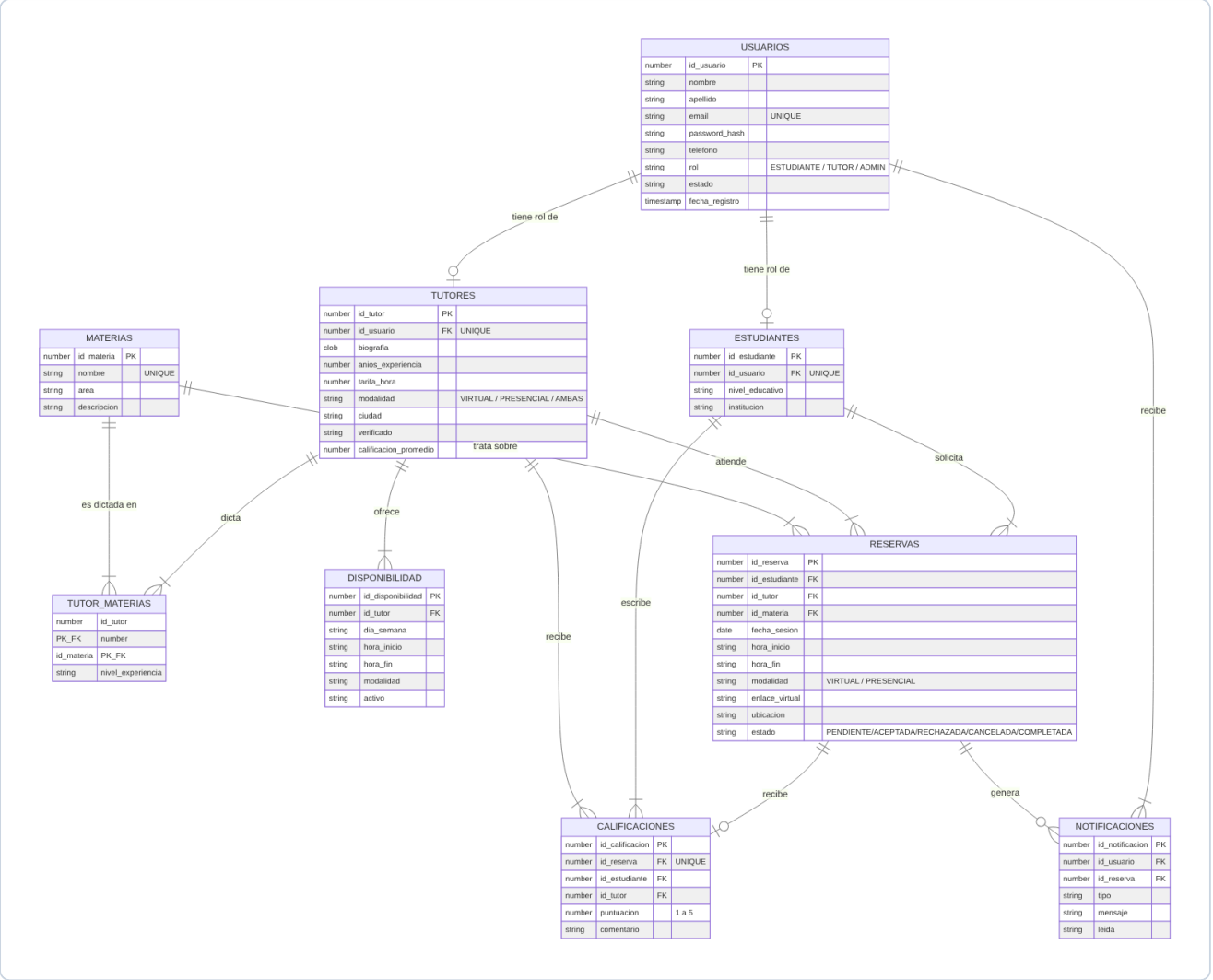
Pendiente de publicacion. El proyecto incluye un workflow de GitHub Actions (`.github/workflows/deploy-pages.yml`) que publica automaticamente el frontend en GitHub Pages en cuanto el repositorio active esa opcion y se fusione esta rama a `main`. El backend tiene un blueprint listo para Render.com (`backend/render.yaml`). Ver `docs/DESPLIEGUE.md` para los pasos exactos.

Una vez publicado, la URL tendra el formato:
`https://<usuario>.github.io/TUTORES_ONLINE_FINAL/`

El frontend funciona en **modo demostracion** (datos en `localStorage`), por lo que sera completamente navegable (registro, login, busqueda de tutores, reservas, calificaciones) sin depender de que el backend este desplegado.

4. Diseño de la base de datos

4.A Diagrama entidad-relacion



4.B Explicacion del modelo

Tabla	Responsabilidad
USUARIOS	Datos comunes de cualquier persona registrada (login, nombre, rol, contacto). Tabla raiz de la jerarquia de roles.
ESTUDIANTES	Extension 1-a-1 de USUARIOS con atributos propios de un estudiante (nivel educativo, institucion).
TUTORES	Extension 1-a-1 de USUARIOS con atributos propios de un tutor (biografia, tarifa, experiencia, verificacion).

MATERIAS	Catalogo de asignaturas que pueden enseñarse.
TUTOR_MATERIAS	Relacion muchos-a-muchos entre TUTORES y MATERIAS (especialidades del tutor).
DISPONIBILIDAD	Franjas horarias que un tutor ofrece, por dia de la semana y modalidad.
RESERVAS	Solicitudes/reservas de tutoria entre un estudiante y un tutor, con estado y modalidad.
CALIFICACIONES	Reseña y puntuacion que un estudiante deja sobre una reserva ya completada.
NOTIFICACIONES	Avisos dirigidos a un usuario sobre cambios en sus reservas.

Decisiones de diseño clave:

- Se separo USUARIOS de ESTUDIANTES/TUTORES (en vez de una sola tabla con columnas nulas segun el rol) para evitar atributos vacios y mantener la integridad referencial de RESERVAS apuntando siempre a ESTUDIANTES/TUTORES.
- Las relaciones muchos-a-muchos (TUTOR_MATERIAS) usan tabla puente con clave compuesta para evitar duplicar especialidades.
- Los estados (RESERVAS.ESTADO, USUARIOS.ROL, etc.) usan restricciones CHECK en vez de tablas de lookup separadas porque son listas cerradas y estables.
- CALIFICACIONES tiene FK unica a RESERVAS (una calificacion por reserva) para impedir calificaciones duplicadas.
- Un indice unico sobre tutor+fecha+hora para las reservas activas previene choques de horario a nivel de base de datos, no solo de aplicacion.

4.C Script SQL — 01_create_tables.sql (creacion de tablas, PK, FK y restricciones)

```
-- ================================================================
-- Tutores On-Line | Modelo de datos – Oracle Database / Oracle APEX
-- Script 1 de 4: creacion de tablas, claves primarias, foraneas y CHECK
-- Compatible con Oracle 12c+ (IDENTITY columns)
-- ================================================================

-- ----------------------------------------------------------------
-- 1. USUARIOS: tabla raiz de cualquier persona registrada en la plataforma
-- ----------------------------------------------------------------
CREATE TABLE usuarios (
    id_usuario       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre           VARCHAR2(80)  NOT NULL,
    apellido         VARCHAR2(80)  NOT NULL,
    email            VARCHAR2(150) NOT NULL,
    password_hash    VARCHAR2(255) NOT NULL,
    telefono         VARCHAR2(20),
    rol              VARCHAR2(10)  NOT NULL,
    estado           VARCHAR2(10)  DEFAULT 'ACTIVO' NOT NULL,
    foto_url         VARCHAR2(255),
    fecha_registro   TIMESTAMP     DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT uq_usuarios_email UNIQUE (email),
    CONSTRAINT ck_usuarios_rol   CHECK (rol IN ('ESTUDIANTE', 'TUTOR', 'ADMIN')),
    CONSTRAINT ck_usuarios_estado CHECK (estado IN ('ACTIVO', 'INACTIVO', 'SUSPENDIDO'))
);

COMMENT ON TABLE usuarios IS 'Cuenta base de cualquier persona registrada (estudiante, tutor o
admin).';
COMMENT ON COLUMN usuarios.password_hash IS 'Hash bcrypt de la contrasena. Nunca se almacena en
texto plano.';

-- ----------------------------------------------------------------
-- 2. ESTUDIANTES: extension 1-a-1 de USUARIOS con rol ESTUDIANTE
-- ----------------------------------------------------------------
CREATE TABLE estudiantes (
    id_estudiante     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_usuario        NUMBER NOT NULL,
    nivel_educativo   VARCHAR2(40),
    institucion       VARCHAR2(120),
    CONSTRAINT uq_estudiantes_usuario UNIQUE (id_usuario),
    CONSTRAINT fk_estudiantes_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,
    CONSTRAINT ck_estudiantes_nivel CHECK (
        nivel_educativo IN ('COLEGIO', 'PREGRADO', 'POSGRADO', 'OTRO') OR nivel_educativo IS NULL
    )
);

COMMENT ON TABLE estudiantes IS 'Atributos propios de un usuario con rol estudiante.';

-- ----------------------------------------------------------------
-- 3. TUTORES: extension 1-a-1 de USUARIOS con rol TUTOR
-- ----------------------------------------------------------------
CREATE TABLE tutores (
    id_tutor              NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_usuario            NUMBER NOT NULL,
    biografia             CLOB,
    anios_experiencia     NUMBER(3) DEFAULT 0,
```

```
    tarifa_hora           NUMBER(10,2) DEFAULT 0,
    modalidad             VARCHAR2(12) DEFAULT 'AMBAS' NOT NULL,
    ciudad                VARCHAR2(80),
    verificado            CHAR(1) DEFAULT 'N' NOT NULL,
    calificacion_promedio NUMBER(3,2) DEFAULT 0,
    total_resenas         NUMBER DEFAULT 0,
    CONSTRAINT uq_tutores_usuario UNIQUE (id_usuario),
    CONSTRAINT fk_tutores_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,
    CONSTRAINT ck_tutores_modalidad CHECK (modalidad IN ('VIRTUAL', 'PRESENCIAL', 'AMBAS')),
    CONSTRAINT ck_tutores_verificado CHECK (verificado IN ('S', 'N')),
    CONSTRAINT ck_tutores_calif CHECK (calificacion_promedio BETWEEN 0 AND 5)
);

COMMENT ON TABLE tutores IS 'Atributos propios de un usuario con rol tutor: perfil, tarifa y
reputacion agregada.';

-- ---------------------------------------------------------------------
-- 4. MATERIAS: catalogo de asignaturas
-- ---------------------------------------------------------------------
CREATE TABLE materias (
    id_materia     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre         VARCHAR2(100) NOT NULL,
    area           VARCHAR2(60),
    descripcion    VARCHAR2(500),
    CONSTRAINT uq_materias_nombre UNIQUE (nombre)
);

COMMENT ON TABLE materias IS 'Catalogo de asignaturas que pueden ser ensenadas en la plataforma.';

-- ---------------------------------------------------------------------
-- 5. TUTOR_MATERIAS: especialidades del tutor (N-a-N tutores <-> materias)
-- ---------------------------------------------------------------------
CREATE TABLE tutor_materias (
    id_tutor            NUMBER NOT NULL,
    id_materia          NUMBER NOT NULL,
    nivel_experiencia   VARCHAR2(15) DEFAULT 'INTERMEDIO',
    CONSTRAINT pk_tutor_materias PRIMARY KEY (id_tutor, id_materia),
    CONSTRAINT fk_tm_tutor FOREIGN KEY (id_tutor)
        REFERENCES tutores (id_tutor) ON DELETE CASCADE,
    CONSTRAINT fk_tm_materia FOREIGN KEY (id_materia)
        REFERENCES materias (id_materia) ON DELETE CASCADE,
    CONSTRAINT ck_tm_nivel CHECK (nivel_experiencia IN ('BASICO', 'INTERMEDIO', 'AVANZADO',
'EXPERTO'))
);

COMMENT ON TABLE tutor_materias IS 'Especialidades: que materias domina cada tutor y en que
nivel.';

-- ---------------------------------------------------------------------
-- 6. DISPONIBILIDAD: franjas horarias que ofrece un tutor
-- ---------------------------------------------------------------------
CREATE TABLE disponibilidad (
    id_disponibilidad   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_tutor            NUMBER NOT NULL,
    dia_semana          VARCHAR2(10) NOT NULL,
    hora_inicio         VARCHAR2(5)  NOT NULL,  -- formato 'HH24:MI'
    hora_fin            VARCHAR2(5)  NOT NULL,
    modalidad           VARCHAR2(12) DEFAULT 'AMBAS' NOT NULL,
    activo              CHAR(1) DEFAULT 'S' NOT NULL,
    CONSTRAINT fk_disp_tutor FOREIGN KEY (id_tutor)
        REFERENCES tutores (id_tutor) ON DELETE CASCADE,
```

```
    CONSTRAINT ck_disp_dia CHECK (dia_semana IN
        ('LUNES','MARTES','MIERCOLES','JUEVES','VIERNES','SABADO','DOMINGO')),
    CONSTRAINT ck_disp_modalidad CHECK (modalidad IN ('VIRTUAL', 'PRESENCIAL', 'AMBAS')),
    CONSTRAINT ck_disp_activo CHECK (activo IN ('S', 'N')),
    CONSTRAINT ck_disp_horas CHECK (hora_inicio < hora_fin)
);

COMMENT ON TABLE disponibilidad IS 'Franjas horarias recurrentes que un tutor ofrece por dia de la
semana.';

-- ---------------------------------------------------------------------
-- 7. RESERVAS: solicitudes/reservas de tutoria
-- ---------------------------------------------------------------------
CREATE TABLE reservas (
    id_reserva          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_estudiante       NUMBER NOT NULL,
    id_tutor            NUMBER NOT NULL,
    id_materia          NUMBER NOT NULL,
    fecha_sesion        DATE         NOT NULL,
    hora_inicio         VARCHAR2(5)  NOT NULL,
    hora_fin            VARCHAR2(5)  NOT NULL,
    modalidad           VARCHAR2(12) NOT NULL,
    enlace_virtual      VARCHAR2(255),
    ubicacion           VARCHAR2(255),
    estado              VARCHAR2(12) DEFAULT 'PENDIENTE' NOT NULL,
    comentario_estudiante VARCHAR2(500),
    motivo_rechazo      VARCHAR2(300),
    fecha_solicitud     TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_res_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiantes (id_estudiante),
    CONSTRAINT fk_res_tutor FOREIGN KEY (id_tutor)
        REFERENCES tutores (id_tutor),
    CONSTRAINT fk_res_materia FOREIGN KEY (id_materia)
        REFERENCES materias (id_materia),
    CONSTRAINT ck_res_modalidad CHECK (modalidad IN ('VIRTUAL', 'PRESENCIAL')),
    CONSTRAINT ck_res_estado CHECK (estado IN
        ('PENDIENTE','ACEPTADA','RECHAZADA','CANCELADA','COMPLETADA')),
    CONSTRAINT ck_res_horas CHECK (hora_inicio < hora_fin)
);

COMMENT ON TABLE reservas IS 'Solicitud de tutoria entre un estudiante y un tutor, con estado y
modalidad.';

-- Evita que un mismo tutor tenga dos reservas activas (pendiente/aceptada) a la misma hora
CREATE UNIQUE INDEX uq_reserva_tutor_horario ON reservas (
    id_tutor, fecha_sesion, hora_inicio,
    CASE WHEN estado IN ('PENDIENTE','ACEPTADA') THEN 1 ELSE NULL END
);

-- ---------------------------------------------------------------------
-- 8. CALIFICACIONES: resena de una reserva ya completada
-- ---------------------------------------------------------------------
CREATE TABLE calificaciones (
    id_calificacion    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_reserva         NUMBER NOT NULL,
    id_estudiante      NUMBER NOT NULL,
    id_tutor           NUMBER NOT NULL,
    puntuacion         NUMBER(2,1) NOT NULL,
    comentario         VARCHAR2(500),
    fecha_creacion     TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT uq_calificaciones_reserva UNIQUE (id_reserva),
    CONSTRAINT fk_calif_reserva FOREIGN KEY (id_reserva)
```

```
        REFERENCES reservas (id_reserva) ON DELETE CASCADE,
    CONSTRAINT fk_calif_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiantes (id_estudiante),
    CONSTRAINT fk_calif_tutor FOREIGN KEY (id_tutor)
        REFERENCES tutores (id_tutor),
    CONSTRAINT ck_calif_puntuacion CHECK (puntuacion BETWEEN 1 AND 5)
);

COMMENT ON TABLE calificaciones IS 'Calificacion y resena que un estudiante deja sobre una reserva
completada.';

-- ---------------------------------------------------------------------
-- 9. NOTIFICACIONES: avisos sobre cambios de estado de una reserva
-- ---------------------------------------------------------------------
CREATE TABLE notificaciones (
    id_notificacion   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_usuario        NUMBER NOT NULL,
    id_reserva        NUMBER,
    tipo              VARCHAR2(30) NOT NULL,
    mensaje           VARCHAR2(300) NOT NULL,
    leida             CHAR(1) DEFAULT 'N' NOT NULL,
    fecha_creacion    TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_notif_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_notif_reserva FOREIGN KEY (id_reserva)
        REFERENCES reservas (id_reserva) ON DELETE CASCADE,
    CONSTRAINT ck_notif_leida CHECK (leida IN ('S', 'N'))
);

COMMENT ON TABLE notificaciones IS 'Avisos dirigidos a un usuario sobre eventos de sus reservas.';
```

4.D Script SQL — 02_indexes.sql (indices de apoyo a busqueda)

```
-- ========================================================================
-- Tutores On-Line | Script 2 de 4: indices de apoyo a busqueda/filtrado
-- Los indices de claves unicas e identity ya se crean automaticamente
-- con las restricciones del script 01; aqui solo se agregan los que
-- aceleran las consultas mas frecuentes de la aplicacion.
-- ========================================================================

-- Busqueda de tutores por ciudad (filtro de modalidad presencial)
CREATE INDEX ix_tutores_ciudad ON tutores (ciudad);

-- Ordenar/filtrar tutores por calificacion (listado "mejor calificados")
CREATE INDEX ix_tutores_calificacion ON tutores (calificacion_promedio);

-- Busqueda de especialidades por materia (listado de tutores que ensenan X materia)
CREATE INDEX ix_tutor_materias_materia ON tutor_materias (id_materia);

-- Consultar disponibilidad de un tutor por dia
CREATE INDEX ix_disponibilidad_tutor_dia ON disponibilidad (id_tutor, dia_semana);

-- Historial de reservas por estudiante y por tutor (dashboards)
CREATE INDEX ix_reservas_estudiante ON reservas (id_estudiante, estado);
CREATE INDEX ix_reservas_tutor ON reservas (id_tutor, estado);
CREATE INDEX ix_reservas_fecha ON reservas (fecha_sesion);

-- Notificaciones no leidas de un usuario (campanita de notificaciones)
CREATE INDEX ix_notificaciones_usuario ON notificaciones (id_usuario, leida);
```

4.E Script SQL — 03_sample_data.sql (datos de prueba)

```
-- ========================================================================
-- Tutores On-Line | Script 3 de 4: datos de prueba coherentes
-- Las contrasenas de prueba son el hash bcrypt de "Tutores2025*"
-- (10 rounds) - unicamente para fines de demostracion academica.
-- ========================================================================

-- ------------------------------------------------------------------------
-- MATERIAS
-- ------------------------------------------------------------------------
INSERT INTO materias (nombre, area, descripcion) VALUES ('Calculo I', 'Matematicas', 'Limites,
derivadas e integrales basicas');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Algebra Lineal', 'Matematicas',
'Matrices, vectores y espacios vectoriales');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Fisica Mecanica', 'Fisica', 'Cinematica,
dinamica y energia');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Circuitos Electricos I', 'Ingenieria
Electrica', 'Analisis de circuitos DC y AC');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Programacion en Python', 'Informatica',
'Fundamentos de programacion y estructuras de datos');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Ingles Conversacional', 'Idiomas',
'Practica de conversacion e inglés general');
INSERT INTO materias (nombre, area, descripcion) VALUES ('Quimica General', 'Quimica',
'Estequiometria, enlaces y reacciones');

-- ------------------------------------------------------------------------
-- USUARIOS (2 tutores + 2 estudiantes + 1 admin)
-- password_hash = bcrypt('Tutores2025*')
```

```
-- ----------------------------------------------------------------------
INSERT INTO usuarios (nombre, apellido, email, password_hash, telefono, rol) VALUES
 ('Laura', 'Martinez', 'laura.martinez@tutoresonline.com',
'$2b$10$e0kq8n0G8m5s0m1F4m5s0eQe0m1F4m5s0eQe0m1F4m5s0eQe0m1Fa', '3001234567', 'TUTOR');
INSERT INTO usuarios (nombre, apellido, email, password_hash, telefono, rol) VALUES
 ('Carlos', 'Ramirez', 'carlos.ramirez@tutoresonline.com',
'$2b$10$e0kq8n0G8m5s0m1F4m5s0eQe0m1F4m5s0eQe0m1F4m5s0eQe0m1Fa', '3007654321', 'TUTOR');
INSERT INTO usuarios (nombre, apellido, email, password_hash, telefono, rol) VALUES
 ('Juan', 'Perez', 'juan.perez@estudiante.com',
'$2b$10$e0kq8n0G8m5s0m1F4m5s0eQe0m1F4m5s0eQe0m1F4m5s0eQe0m1Fa', '3109876543', 'ESTUDIANTE');
INSERT INTO usuarios (nombre, apellido, email, password_hash, telefono, rol) VALUES
 ('Maria', 'Gomez', 'maria.gomez@estudiante.com',
'$2b$10$e0kq8n0G8m5s0m1F4m5s0eQe0m1F4m5s0eQe0m1F4m5s0eQe0m1Fa', '3151122334', 'ESTUDIANTE');
INSERT INTO usuarios (nombre, apellido, email, password_hash, telefono, rol) VALUES
 ('Admin', 'Plataforma', 'admin@tutoresonline.com',
'$2b$10$e0kq8n0G8m5s0m1F4m5s0eQe0m1F4m5s0eQe0m1F4m5s0eQe0m1Fa', NULL, 'ADMIN');

-- ----------------------------------------------------------------------
-- ESTUDIANTES (referencian a USUARIOS por email -> subquery)
-- ----------------------------------------------------------------------
INSERT INTO estudiantes (id_usuario, nivel_educativo, institucion)
 SELECT id_usuario, 'PREGRADO', 'Corporacion Universitaria de la Costa (CUC)' FROM usuarios WHERE
email = 'juan.perez@estudiante.com';
INSERT INTO estudiantes (id_usuario, nivel_educativo, institucion)
 SELECT id_usuario, 'COLEGIO', 'Institucion Educativa San Jose' FROM usuarios WHERE email =
'maria.gomez@estudiante.com';

-- ----------------------------------------------------------------------
-- TUTORES
-- ----------------------------------------------------------------------
INSERT INTO tutores (id_usuario, biografia, anios_experiencia, tarifa_hora, modalidad, ciudad,
verificado, calificacion_promedio, total_resenas)
 SELECT id_usuario, 'Ingeniera con 5 anios de experiencia enseniando matematicas y fisica a nivel
universitario.', 5, 35000, 'AMBAS', 'Barranquilla', 'S', 4.8, 12
 FROM usuarios WHERE email = 'laura.martinez@tutoresonline.com';
INSERT INTO tutores (id_usuario, biografia, anios_experiencia, tarifa_hora, modalidad, ciudad,
verificado, calificacion_promedio, total_resenas)
 SELECT id_usuario, 'Tutor especializado en programacion e ingenieria electrica, clases practicas
y orientadas a proyectos.', 3, 30000, 'VIRTUAL', 'Barranquilla', 'S', 4.6, 8
 FROM usuarios WHERE email = 'carlos.ramirez@tutoresonline.com';

-- ----------------------------------------------------------------------
-- TUTOR_MATERIAS (especialidades)
-- ----------------------------------------------------------------------
INSERT INTO tutor_materias (id_tutor, id_materia, nivel_experiencia)
 SELECT t.id_tutor, m.id_materia, 'EXPERTO' FROM tutores t, usuarios u, materias m
 WHERE t.id_usuario = u.id_usuario AND u.email = 'laura.martinez@tutoresonline.com' AND m.nombre =
'Calculo I';
INSERT INTO tutor_materias (id_tutor, id_materia, nivel_experiencia)
 SELECT t.id_tutor, m.id_materia, 'AVANZADO' FROM tutores t, usuarios u, materias m
 WHERE t.id_usuario = u.id_usuario AND u.email = 'laura.martinez@tutoresonline.com' AND m.nombre =
'Algebra Lineal';
INSERT INTO tutor_materias (id_tutor, id_materia, nivel_experiencia)
 SELECT t.id_tutor, m.id_materia, 'EXPERTO' FROM tutores t, usuarios u, materias m
 WHERE t.id_usuario = u.id_usuario AND u.email = 'laura.martinez@tutoresonline.com' AND m.nombre =
'Fisica Mecanica';
INSERT INTO tutor_materias (id_tutor, id_materia, nivel_experiencia)
 SELECT t.id_tutor, m.id_materia, 'EXPERTO' FROM tutores t, usuarios u, materias m
 WHERE t.id_usuario = u.id_usuario AND u.email = 'carlos.ramirez@tutoresonline.com' AND m.nombre =
'Programacion en Python';
INSERT INTO tutor_materias (id_tutor, id_materia, nivel_experiencia)
 SELECT t.id_tutor, m.id_materia, 'AVANZADO' FROM tutores t, usuarios u, materias m
```

```
 WHERE t.id_usuario = u.id_usuario AND u.email = 'carlos.ramirez@tutoresonline.com' AND m.nombre =
'Circuitos Electricos I';

-- ------------------------------------------------------------------
-- DISPONIBILIDAD
-- ------------------------------------------------------------------
INSERT INTO disponibilidad (id_tutor, dia_semana, hora_inicio, hora_fin, modalidad)
 SELECT t.id_tutor, 'LUNES', '15:00', '18:00', 'AMBAS' FROM tutores t, usuarios u
 WHERE t.id_usuario = u.id_usuario AND u.email = 'laura.martinez@tutoresonline.com';
INSERT INTO disponibilidad (id_tutor, dia_semana, hora_inicio, hora_fin, modalidad)
 SELECT t.id_tutor, 'MIERCOLES', '15:00', '18:00', 'VIRTUAL' FROM tutores t, usuarios u
 WHERE t.id_usuario = u.id_usuario AND u.email = 'laura.martinez@tutoresonline.com';
INSERT INTO disponibilidad (id_tutor, dia_semana, hora_inicio, hora_fin, modalidad)
 SELECT t.id_tutor, 'MARTES', '18:00', '21:00', 'VIRTUAL' FROM tutores t, usuarios u
 WHERE t.id_usuario = u.id_usuario AND u.email = 'carlos.ramirez@tutoresonline.com';
INSERT INTO disponibilidad (id_tutor, dia_semana, hora_inicio, hora_fin, modalidad)
 SELECT t.id_tutor, 'JUEVES', '18:00', '21:00', 'VIRTUAL' FROM tutores t, usuarios u
 WHERE t.id_usuario = u.id_usuario AND u.email = 'carlos.ramirez@tutoresonline.com';

-- ------------------------------------------------------------------
-- RESERVAS (una pendiente, una aceptada futura, una completada pasada)
-- ------------------------------------------------------------------
INSERT INTO reservas (id_estudiante, id_tutor, id_materia, fecha_sesion, hora_inicio, hora_fin,
modalidad, enlace_virtual, estado, comentario_estudiante)
 SELECT e.id_estudiante, t.id_tutor, m.id_materia, TRUNC(SYSDATE) + 3, '15:00', '16:00',
'VIRTUAL',
        'https://meet.tutoresonline.com/sala-demo-1', 'PENDIENTE', 'Necesito ayuda con derivadas
para el parcial.'
 FROM estudiantes e, usuarios ue, tutores t, usuarios ut, materias m
 WHERE e.id_usuario = ue.id_usuario AND ue.email = 'juan.perez@estudiante.com'
   AND t.id_usuario = ut.id_usuario AND ut.email = 'laura.martinez@tutoresonline.com'
   AND m.nombre = 'Calculo I';

INSERT INTO reservas (id_estudiante, id_tutor, id_materia, fecha_sesion, hora_inicio, hora_fin,
modalidad, enlace_virtual, estado)
 SELECT e.id_estudiante, t.id_tutor, m.id_materia, TRUNC(SYSDATE) + 7, '18:00', '19:00',
'VIRTUAL',
        'https://meet.tutoresonline.com/sala-demo-2', 'ACEPTADA'
 FROM estudiantes e, usuarios ue, tutores t, usuarios ut, materias m
 WHERE e.id_usuario = ue.id_usuario AND ue.email = 'maria.gomez@estudiante.com'
   AND t.id_usuario = ut.id_usuario AND ut.email = 'carlos.ramirez@tutoresonline.com'
   AND m.nombre = 'Programacion en Python';

INSERT INTO reservas (id_estudiante, id_tutor, id_materia, fecha_sesion, hora_inicio, hora_fin,
modalidad, estado)
 SELECT e.id_estudiante, t.id_tutor, m.id_materia, TRUNC(SYSDATE) - 10, '15:00', '16:00',
'PRESENCIAL', 'COMPLETADA'
 FROM estudiantes e, usuarios ue, tutores t, usuarios ut, materias m
 WHERE e.id_usuario = ue.id_usuario AND ue.email = 'juan.perez@estudiante.com'
   AND t.id_usuario = ut.id_usuario AND ut.email = 'laura.martinez@tutoresonline.com'
   AND m.nombre = 'Fisica Mecanica';

-- Ubicacion para la reserva presencial completada
UPDATE reservas SET ubicacion = 'Biblioteca CUC, sala de estudio 4'
 WHERE modalidad = 'PRESENCIAL' AND estado = 'COMPLETADA';

-- ------------------------------------------------------------------
-- CALIFICACIONES (solo sobre la reserva COMPLETADA)
-- ------------------------------------------------------------------
INSERT INTO calificaciones (id_reserva, id_estudiante, id_tutor, puntuacion, comentario)
 SELECT r.id_reserva, r.id_estudiante, r.id_tutor, 5, 'Excelente explicacion, muy paciente y
clara.'
```

```
 FROM reservas r WHERE r.estado = 'COMPLETADA';

-- ----------------------------------------------------------------------
-- NOTIFICACIONES
-- ----------------------------------------------------------------------
INSERT INTO notificaciones (id_usuario, id_reserva, tipo, mensaje)
 SELECT ut.id_usuario, r.id_reserva, 'NUEVA_SOLICITUD', 'Tienes una nueva solicitud de tutoria
pendiente por responder.'
 FROM reservas r
 JOIN tutores t ON t.id_tutor = r.id_tutor
 JOIN usuarios ut ON ut.id_usuario = t.id_usuario
 WHERE r.estado = 'PENDIENTE';

INSERT INTO notificaciones (id_usuario, id_reserva, tipo, mensaje)
 SELECT ue.id_usuario, r.id_reserva, 'RESERVA_ACEPTADA', 'Tu tutoria fue aceptada. Revisa la fecha
y el enlace de la sesion.'
 FROM reservas r
 JOIN estudiantes e ON e.id_estudiante = r.id_estudiante
 JOIN usuarios ue ON ue.id_usuario = e.id_usuario
 WHERE r.estado = 'ACEPTADA';

COMMIT;
```

4.F Script SQL — 04_validation_queries.sql (consultas de validacion)

```
-- ========================================================================
-- Tutores On-Line | Script 4 de 4: consultas de validacion del modelo
-- Ejecutar despues de 03_sample_data.sql para comprobar que las
-- relaciones, restricciones y datos funcionan correctamente.
-- ========================================================================

-- 1) Conteo de filas por tabla (ninguna debe quedar en 0 tras los datos de prueba)
SELECT 'USUARIOS' tabla, COUNT(*) filas FROM usuarios
UNION ALL SELECT 'ESTUDIANTES', COUNT(*) FROM estudiantes
UNION ALL SELECT 'TUTORES', COUNT(*) FROM tutores
UNION ALL SELECT 'MATERIAS', COUNT(*) FROM materias
UNION ALL SELECT 'TUTOR_MATERIAS', COUNT(*) FROM tutor_materias
UNION ALL SELECT 'DISPONIBILIDAD', COUNT(*) FROM disponibilidad
UNION ALL SELECT 'RESERVAS', COUNT(*) FROM reservas
UNION ALL SELECT 'CALIFICACIONES', COUNT(*) FROM calificaciones
UNION ALL SELECT 'NOTIFICACIONES', COUNT(*) FROM notificaciones;

-- 2) Listado de tutores con su usuario, materias y calificacion (vista "buscar tutores")
SELECT u.nombre || ' ' || u.apellido AS tutor,
       t.ciudad, t.modalidad, t.tarifa_hora, t.calificacion_promedio,
       LISTAGG(m.nombre, ', ') WITHIN GROUP (ORDER BY m.nombre) AS especialidades
FROM tutores t
JOIN usuarios u ON u.id_usuario = t.id_usuario
JOIN tutor_materias tm ON tm.id_tutor = t.id_tutor
JOIN materias m ON m.id_materia = tm.id_materia
GROUP BY u.nombre, u.apellido, t.ciudad, t.modalidad, t.tarifa_hora, t.calificacion_promedio
ORDER BY t.calificacion_promedio DESC;

-- 3) Disponibilidad semanal de un tutor especifico
SELECT u.nombre || ' ' || u.apellido AS tutor, d.dia_semana, d.hora_inicio, d.hora_fin,
d.modalidad
FROM disponibilidad d
JOIN tutores t ON t.id_tutor = d.id_tutor
JOIN usuarios u ON u.id_usuario = t.id_usuario
WHERE u.email = 'laura.martinez@tutoresonline.com'
ORDER BY d.dia_semana;

-- 4) Reservas con todos sus datos legibles (estudiante, tutor, materia, estado)
SELECT r.id_reserva,
       ue.nombre || ' ' || ue.apellido AS estudiante,
       ut.nombre || ' ' || ut.apellido AS tutor,
       m.nombre AS materia,
       r.fecha_sesion, r.hora_inicio, r.hora_fin, r.modalidad, r.estado
FROM reservas r
JOIN estudiantes e ON e.id_estudiante = r.id_estudiante
JOIN usuarios ue ON ue.id_usuario = e.id_usuario
JOIN tutores t ON t.id_tutor = r.id_tutor
JOIN usuarios ut ON ut.id_usuario = t.id_usuario
JOIN materias m ON m.id_materia = r.id_materia
ORDER BY r.fecha_sesion;

-- 5) Historial y calificaciones de un estudiante
SELECT r.id_reserva, m.nombre AS materia, r.fecha_sesion, r.estado,
       c.puntuacion, c.comentario
FROM reservas r
JOIN estudiantes e ON e.id_estudiante = r.id_estudiante
JOIN usuarios ue ON ue.id_usuario = e.id_usuario
JOIN materias m ON m.id_materia = r.id_materia
```

```
LEFT JOIN calificaciones c ON c.id_reserva = r.id_reserva
WHERE ue.email = 'juan.perez@estudiante.com'
ORDER BY r.fecha_sesion DESC;

-- 6) Notificaciones no leidas de un usuario
SELECT n.tipo, n.mensaje, n.fecha_creacion
FROM notificaciones n
JOIN usuarios u ON u.id_usuario = n.id_usuario
WHERE u.email = 'carlos.ramirez@tutoresonline.com' AND n.leida = 'N'
ORDER BY n.fecha_creacion DESC;

-- 7) Verificacion de integridad referencial: debe devolver 0 filas en todos los casos
SELECT COUNT(*) huerfanos_reservas_estudiante FROM reservas r
 WHERE NOT EXISTS (SELECT 1 FROM estudiantes e WHERE e.id_estudiante = r.id_estudiante);
SELECT COUNT(*) huerfanos_reservas_tutor FROM reservas r
 WHERE NOT EXISTS (SELECT 1 FROM tutores t WHERE t.id_tutor = r.id_tutor);
SELECT COUNT(*) huerfanos_calificaciones FROM calificaciones c
 WHERE NOT EXISTS (SELECT 1 FROM reservas r WHERE r.id_reserva = c.id_reserva);
```

5. Resumen de funcionalidades implementadas

✅ Implementado y probado ⏳ Pendiente de Oracle real

Funcionalidad	Estado
Pagina de inicio, navegacion y diseño responsive	✅ Implementado y probado
Registro de estudiantes y tutores con validaciones	✅ Implementado y probado
Login / logout con contraseñas hasheadas (bcrypt) y JWT en backend	✅ Implementado y probado
Busqueda de tutores con filtros (materia, modalidad, ciudad) y orden	✅ Implementado y probado
Perfil detallado de tutor (especialidades, disponibilidad, reseñas)	✅ Implementado y probado
Reserva de tutoria con seleccion de fecha/hora y prevencion de conflictos	✅ Implementado y probado
Gestion de solicitudes por el tutor (aceptar / rechazar con motivo)	✅ Implementado y probado
Gestion de disponibilidad semanal del tutor	✅ Implementado y probado
Calificaciones y reseñas tras tutoria completada	✅ Implementado y probado
Notificaciones de cambios de estado de reserva	✅ Implementado y probado
API REST (Node/Express) con JWT, validaciones y manejo de errores	✅ Implementado y probado (sin Oracle conectado)
Conexion real a Oracle Database / Oracle APEX	⏳ Pendiente de una instancia Oracle real
Videollamada integrada (solo se captura el enlace, no hay integracion real)	⏳ Diseñado, no integrado con un proveedor real
Despliegue publico (GitHub Pages / Render)	⏳ Pendiente de accion manual (ver seccion 3)

6. Tecnologias utilizadas

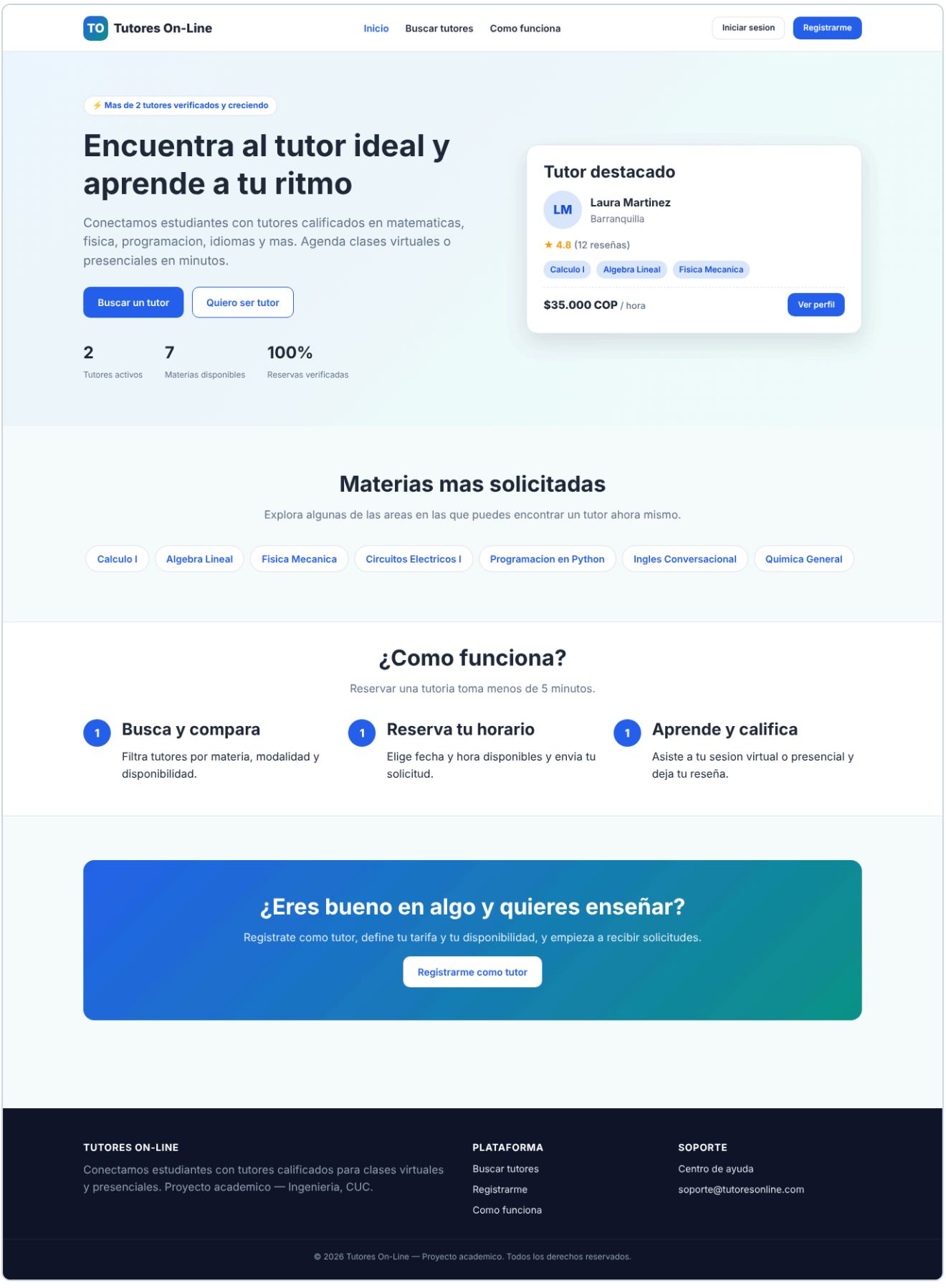
- **Frontend:** HTML5, CSS3 (variables, componentes, diseño responsive), JavaScript ES6+ (sin frameworks ni build tools).
- **Backend:** Node.js + Express (API REST), JWT para autenticacion, bcryptjs para hash de contraseñas.

- **Base de datos:** Oracle Database / Oracle APEX (Oracle SQL), driver node-oracledb en modo "thin" (sin cliente nativo).
- **Pruebas:** Node.js test runner nativo (utilidades y rutas HTTP), Playwright (pruebas de navegacion end-to-end).
- **Despliegue previsto:** GitHub Pages (frontend) + Render.com (backend).

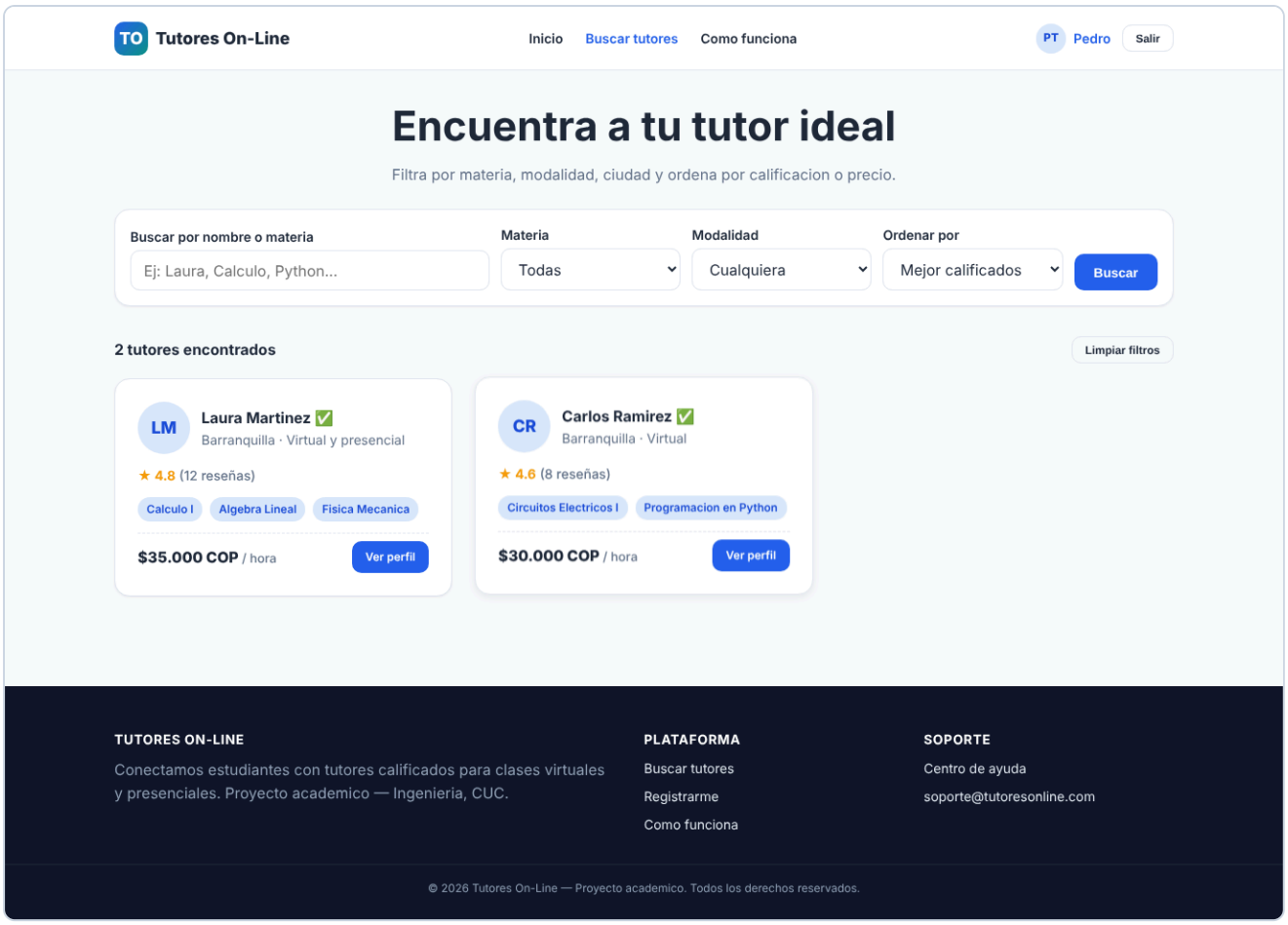
7. Evidencias visuales de las interfaces

Capturas tomadas con un navegador real (Chromium) durante las pruebas automatizadas de navegacion end-to-end.

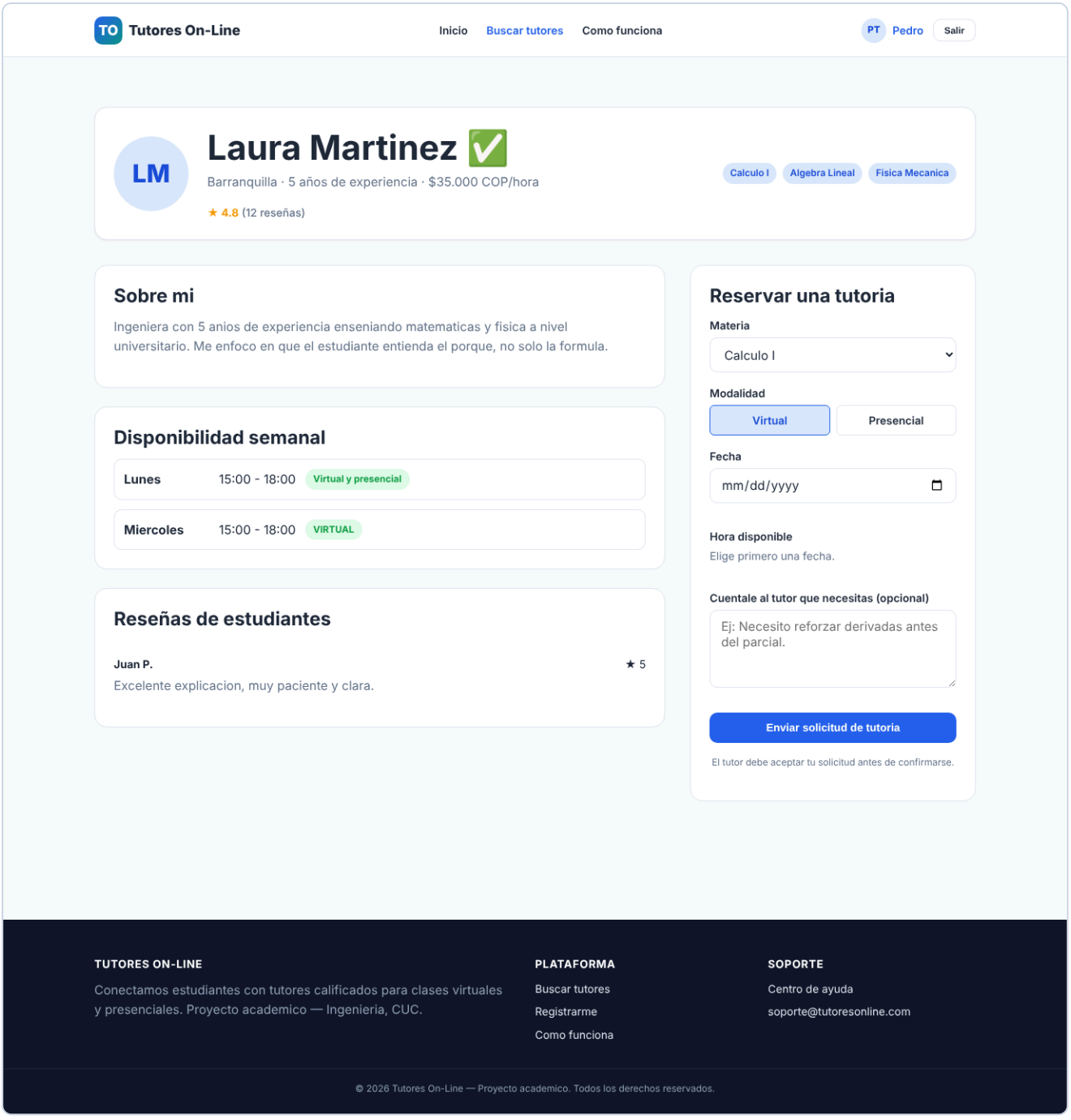
Pagina de inicio



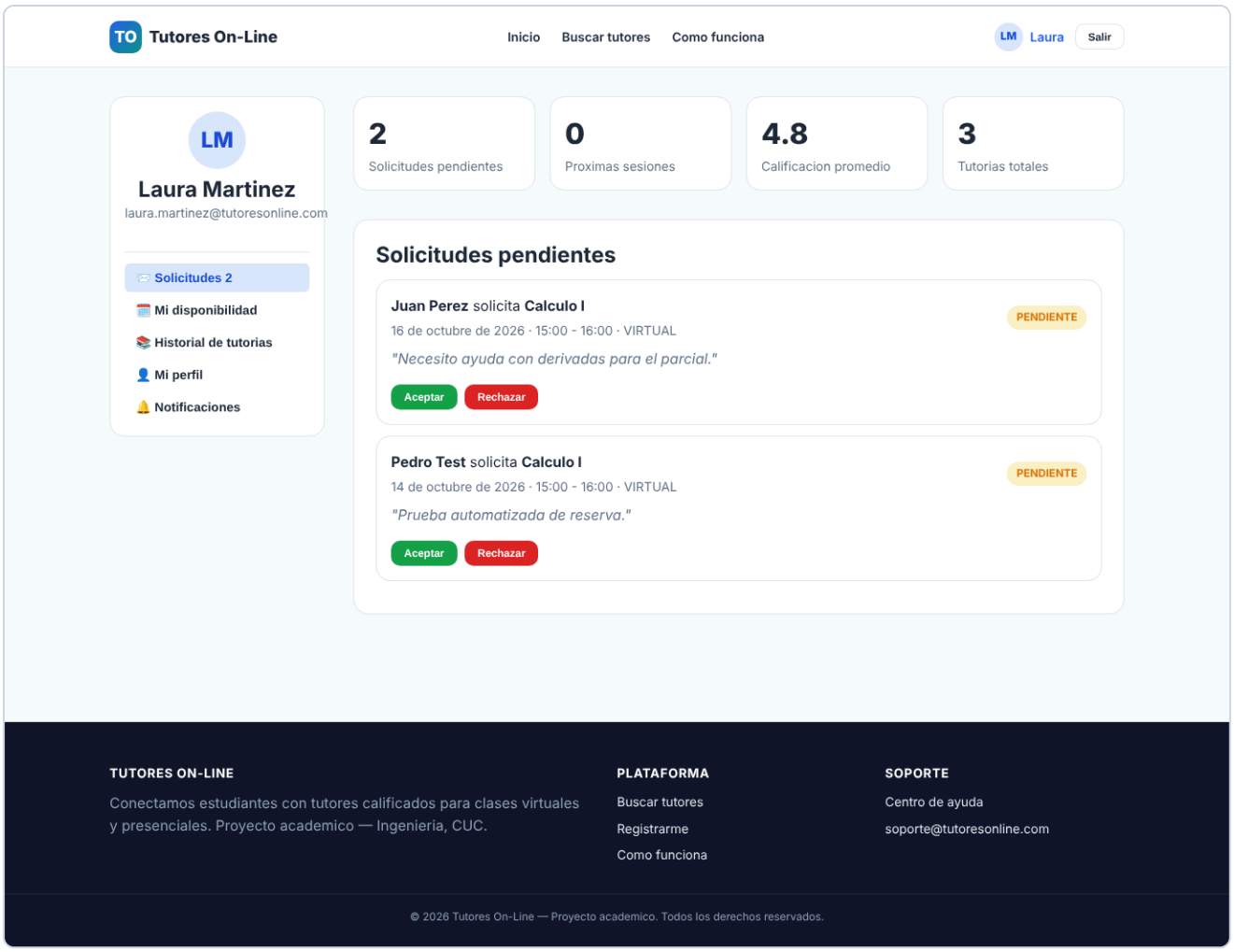
Busqueda de tutores con filtros



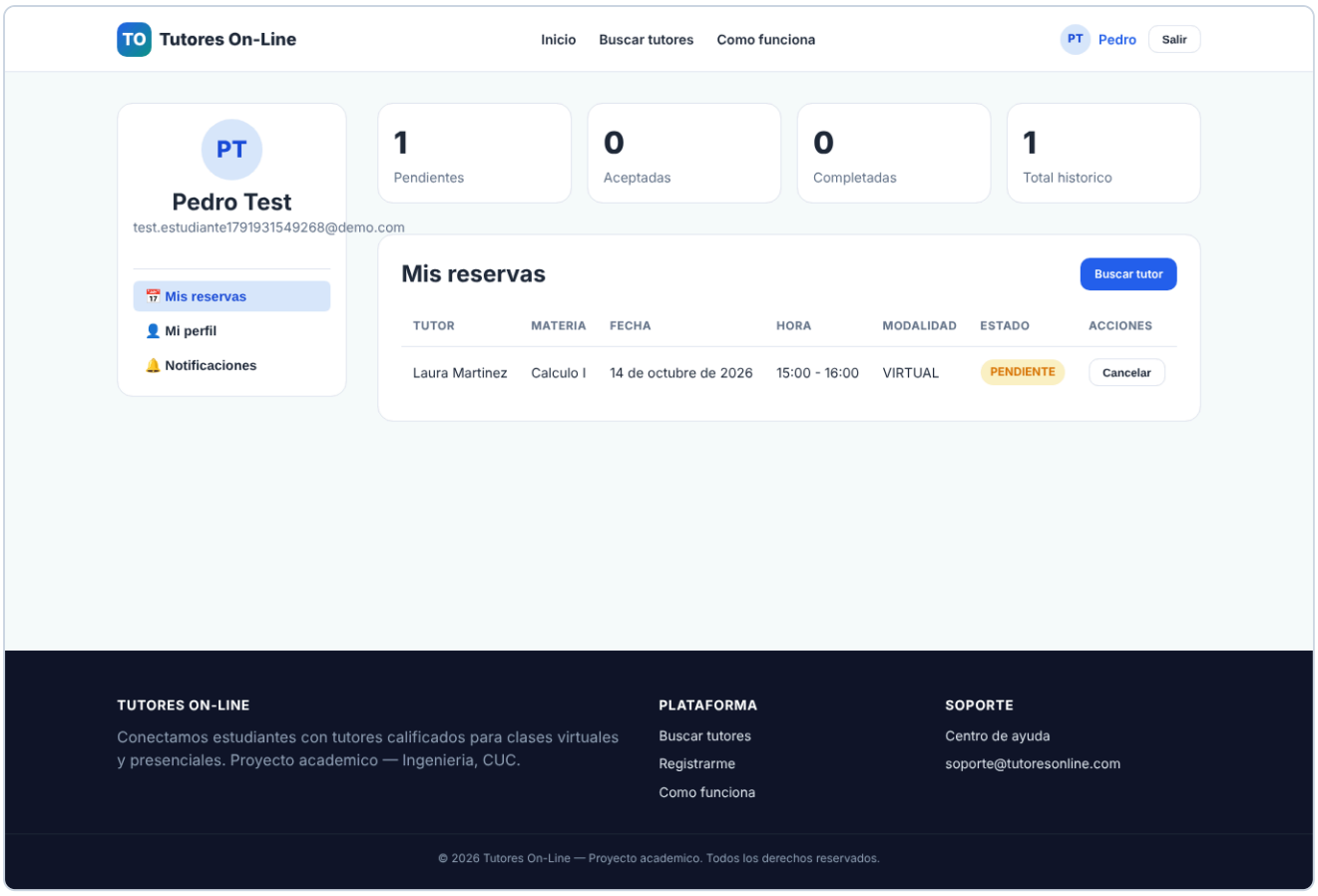
Perfil de tutor y formulario de reserva



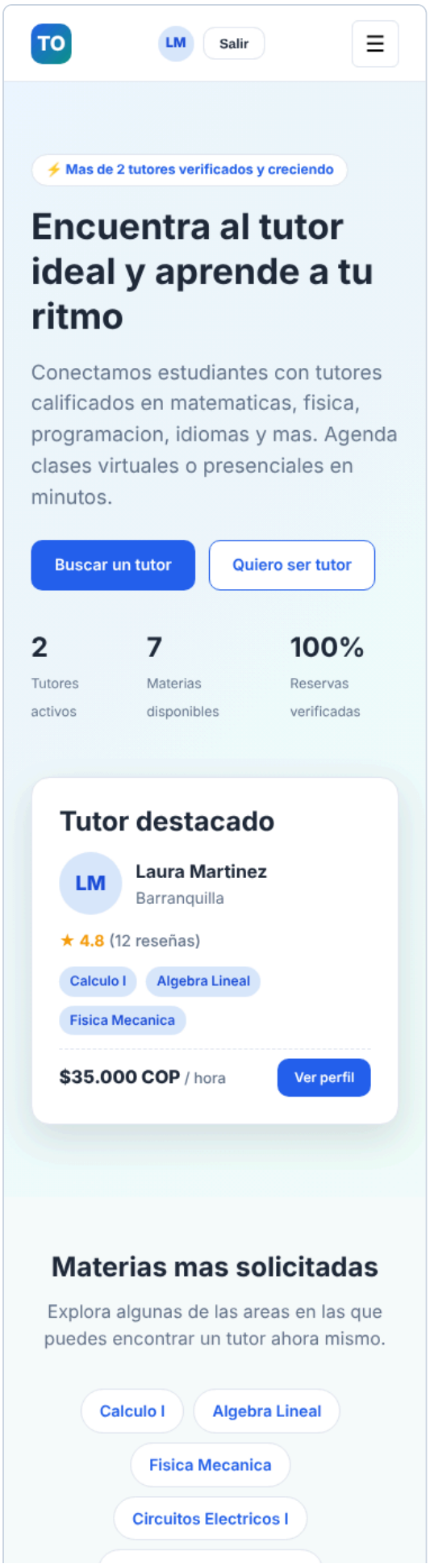
Panel del tutor (solicitudes pendientes)



Panel del estudiante (mis reservas)



Diseño responsive (vista movil, 375px)


TO
LM
Salir
⚡ Mas de 2 tutores verificados y creciendo
Encuentra al tutor ideal y aprende a tu ritmo
Conectamos estudiantes con tutores calificados en matematicas, fisica, programacion, idiomas y mas. Agenda clases virtuales o presenciales en minutos.
Buscar un tutor
Quiero ser tutor
2
Tutores activos
7
Materias disponibles
100%
Reservas verificadas
Tutor destacado
LM
Laura Martinez
Barranquilla
★ 4.8 (12 reseñas)
Calculo I
Algebra Lineal
Fisica Mecanica
$35.000 COP / hora
Ver perfil
Materias mas solicitadas
Explora algunas de las areas en las que puedes encontrar un tutor ahora mismo.
Calculo I
Algebra Lineal
Fisica Mecanica
Circuitos Electricos I

Programacion en Python

Ingles Conversacional

Quimica General

¿Como funciona?

Reservar una tutoria toma menos de 5 minutos.

1 **Busca y compara**

Filtra tutores por materia, modalidad y disponibilidad.

1 **Reserva tu horario**

Elige fecha y hora disponibles y envia tu solicitud.

1 **Aprende y califica**

Asiste a tu sesion virtual o presencial y deja tu reseña.

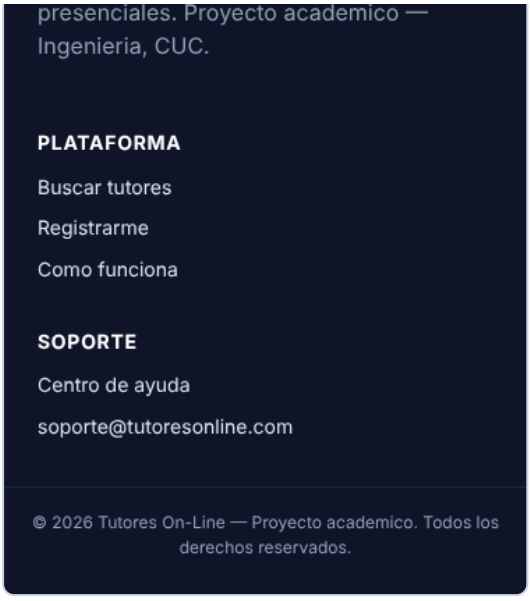
¿Eres bueno en algo y quieres enseñar?

Registrate como tutor, define tu tarifa y tu disponibilidad, y empieza a recibir solicitudes.

Registrarme como tutor

TUTORES ON-LINE

Conectamos estudiantes con tutores calificados para clases virtuales y



8. Estado de las pruebas

Pruebas ejecutadas y exitosas:

- **Pruebas unitarias del backend** (`tests/backend-utils.test.js`): hash y verificacion de contraseñas (bcrypt), firma y verificacion de JWT. 4/4 pruebas pasadas.
- **Pruebas de integracion HTTP** (`tests/backend-routes.test.js`): health check, manejo de rutas inexistentes (404), autenticacion requerida (401), validacion de datos de entrada (400), y respuesta controlada (503, no 500) cuando Oracle no esta configurado. 6/6 pruebas pasadas.
- **Pruebas de navegacion end-to-end** (Playwright, navegador real): flujo completo registro → busqueda → reserva → login como tutor → aceptar solicitud → gestionar disponibilidad, sin errores de consola. Se recorrieron las 9 paginas del sitio verificando que cargan sin errores.
- **Prueba de diseño responsive:** verificada visualmente en viewport de escritorio (1366px) y movil (375px).

Pendiente: pruebas contra una instancia Oracle real (requiere que se configure la conexion descrita en la seccion 3 de `docs/DESPLIEGUE.md`).

9. Estado real del despliegue

Componente	Estado
Codigo fuente en GitHub	✅ Publicado en la rama de trabajo
Workflow de despliegue (GitHub Pages)	✅ Implementado, pendiente de activacion manual

Frontend publicado y accesible	⏳ Pendiente de que se active GitHub Pages (ver seccion 3)
Backend publicado y accesible	⏳ Pendiente de cuenta de hosting (ver docs/DESPLIEGUE.md)
Oracle Database conectada	⏳ Pendiente de instancia Oracle APEX real

10. Instrucciones breves de ejecucion

```
Frontend (modo demostracion, sin backend):
  npx http-server frontend -p 8080
  abrir http://localhost:8080

Backend:
  cd backend
  cp .env.example .env   (completar credenciales de Oracle)
  npm install
  npm run dev             (http://localhost:3000)

Pruebas automatizadas:
  npm test                (en la raiz del repositorio)

Base de datos:
  Ejecutar en Oracle APEX SQL Workshop, en orden:
  database/oracle/01_create_tables.sql
  database/oracle/02_indexes.sql
  database/oracle/03_sample_data.sql
  database/oracle/04_validation_queries.sql
```

11. Limitaciones y funcionalidades pendientes

- No hay integracion real con un proveedor de videollamadas (Zoom/Meet): solo se captura y guarda el enlace que el tutor comparte.
- No hay integracion de notificaciones por correo electronico o WhatsApp (las notificaciones son internas, dentro de la plataforma).
- La conexion a Oracle Database no ha sido probada contra una instancia real (no se contaba con credenciales durante el desarrollo).
- El despliegue publico (GitHub Pages / Render) no se ha activado porque requiere acciones manuales en las cuentas del propietario del repositorio.
- No se implemento un sistema de pagos (fuera del alcance academico definido).